Réseau de neurone

December 2, 2022

`auteur: Mousssa Seydi Faye`
`profession: Ingénieur en mathématique et numérique`

---

1 Introduction

L'idée générale de ce tutoriel,c'est de donnée une compréhension théorique et pratique pour coder un réseau de neurone fonctionnel capable de résoudre quelques problème de machine learning. Nous tenterons de ne faire appel à un aucune bibliothèque spécialise en deep learning ni en auto-diff. On va essayer de palier théorie et pratique pour nous donner une meilleur compréhension des réseaux de neurones et de leurs fonctionnements.

2 Réseau de neurone

2.1 Principe de base

Détaillons chaque phase:

- Partie linéaire :

Un neurone est d'abord muni de poids $w_1, w_2, w_3, ..., w_n$ qui détermine une fonction linéaire

(Uni-dimension).

si l'entrée $x \in \mathcal{R}$ donc le problème se résume a :

$$z = wx + b$$

(Multi-dimension)

si l'entree $X \in \mathcal{R}^n$ avec $X = (x_1, x_2, ..., x_n)$ donc le problème se résume a :

$$z = \sum_{i=1}^{n} w_i x_i \ + \ \theta$$

- Fonction d'activation :

  La sortie linéaire z est ensuite composée par une fonction dite d'activation φ

- Sortie :

  La valeur de sortie est donc

$$a = \varphi(z)$$

Cependant, un neurone est trop élémentaire pour résoudre des problèmes complexes.

Un réseau de neurone est la juxtaposition de plusieurs neurones regroupés dans des couches.

Comme tous les algorithmes d'apprentissage, il faut trouver un moyen de faire apprendre à nos neurones a extraire l'information de nos données, enfin de résoudre un problème bien spécifique (phase d'apprentissage). comme un réseau de neurone peut contenir des millions de paramètres alors trouver les meilleurs paramètres peut être une tache ardue sachant que la modification d'un paramètre peut affecter tous les paramètres du modèle et la sortie prédite.

2.1.1 Importation des bibliothèques (Calcul matricielle)

A partir de notre compréhension théorique des réseaux de neurones artificielles, nous essayerons de traduire le langage mathématique sous forme d'algorithme applicable à des problèmes réels.

Nous voulons codé un réseau de neurone capable prendre autant de couche que possible. On sait que qu'en changeant l'architecture du modèle, nous changeons aussi la formulation du problème.

Donc le but est de faire une implémentation qui fait abstraction de l'architecture du modèle. Ce qui nous permettra de développer notre algorithmique capable plusieurs type de problème et une meilleur compréhension des réseaux de neurone du type DNN ou du perceptron multicouche.

Pour cela nous allons implémenter les couches indépendamment du modèle.

Nous aurons besoin des modules python:

[2]:
```
import sys
try:
    import numpy as np
    import pandas as pd
    import os
    import plotly.express as px
    import plotly.graph_objects as go
    from plotly.subplots import make_subplots
    import time
    from sklearn import datasets
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score, recall_score, f1_score,\
                                ␣
 ↪confusion_matrix,precision_score,mean_absolute_error,mean_squared_error
    from sklearn.preprocessing import MinMaxScaler
    import matplotlib.pyplot as plt
except:
    #%%%%%%%%%%%%%%%%%%%%%%%%%----------------Installation des modules de␣
 ↪travails----------------%%%%%%%%%%%%%%%%%
    # Pour les calculs matricielles
```

```
    !pip install numpy
    # Pour les graphiques
    !pip install plotly
    !pip install matplotlib
    # Pour gérer les données
    !pip install pandas
    # Pour utiliser les données dans le module pour tester notre modèle
    !pip install sklearn


"""
Ces fonctions seront utilisées pour illustré le bon fonctionnement de notre␣
↪modèle
"""

#%%%
```

```
    fig.add_trace(go.Contour(x=xx.ravel(),y=yy.ravel(),z=zz.ravel(),opacity=0.
↪5,showscale=False))
    fig.update_layout(height=height,width=width,template='plotly_white')
    fig.show()
    if return_fig:
        return fig

np.random.seed(1234)
```

[3]:
```
import sklearn
```

2.2 Ce que fait chaque couche neurone et organisation ?

Nous allons définir une nouvelle classe d'objet qu'on va nommer Couche.

[4]:
```
class Couches(object):
    def __init__(self):
        pass
```

Nous remarquons qu'un neurone est définie par une fonction d'activation qu'on va noter σ, ses paramètres qu'on notera ω. Donc comme illustrer ci-dessous on a :

$$\hat{y} = \sigma\left(< x, \omega > + b\right)$$

Pour faciliter le codage du modèle nous allons considérer que sur une couches de neurones, les poids sont représente par la matrice W d'où :

$$W = \begin{bmatrix} \omega_{1,1}, ..., \omega_{1,j} \\ \omega_{2,1}, ..., \omega_{2,j} \\ \vdots, ..., \vdots \\ \omega_{i,1}, ..., \omega_{i,j} \end{bmatrix}$$

ou chaque colonne représente les paramètres d'un neurone. Et soit

$$b = \begin{bmatrix} b_1 \\ \vdots \\ b_i \end{bmatrix}$$

les biais de chaque neurones Nous savons maintenant qu'une couche est définie par la matrice des poids W, des biais b et la fonction d'activation et sa dérivée a défaut de la dérivé analogique; la dérivé numérique sera utilisée.

Donc à notre classe Couches nous allons ajouter un constructeur qui nous permettra d'initialisé tous les couches.

- Entrées :

  Nb_neurone : Le nombre de neurone contenu dans la couche définie

  activation : La fonction d'activation

dactivation : la dérivée de la fonction d'activation

dim_input : la dimension des données d'entrées dans la couche définit par defaut à 2

h : la valeur d'approximation de la dérivée numérique

[5]:
```
def __init__(self,Nb_neurones:int, activation, dactivation=None, dim_input=2,␣
↪h=1e-9):
    self.activation = activation
    self.dactivation = dactivation if dactivation != None else Couches.
↪Derivee(activation,h=h)
    # On va initialiser les paramétres de notre modéle
    # La valeur de w, b, z, a definit dans la théorie des couches de neurone␣
↪(Principe de base)
    self.w = (np.random.randn(Nb_neurones,dim_input)/np.sqrt(dim_input)).
↪astype(np.float64) # poids
    self.b = np.zeros(Nb_neurones).astype(np.float64) # biais
    self.z = None # la partie linéaire de notre sortie
    self.a = None # la valeur d'activation de notre couche (regroupant tous les␣
↪valeurs activations de chaque neurone)
    # Ces paramètres sont relatifs à la méthode RMSprop qui est une variante de␣
↪la descente du gradient
    # On les utilisera pendant la phase d'apprentissage et sont seulement lier à␣
↪cette méthode
    self.vdw = 0
    self.vdb = 0
Couches.__init__ = __init__
```

Si la dérivée analogique n'est pas donnée, nous utiliserons alors la formule d'approximation de la dérivée d'Euler centré définie par :

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h}$$

[6]:
```
def Derivee(fun, h):
    # h va tendre vers 0 et pour stabilite numerique ne doit pas etre egale a 0
    return lambda x : (fun(x+h)-fun(x-h))/(2*h)

Couches.Dérivée = Dérivée
```

Quelque soit la couche que nous codons, il y' aura toujours au moins deux éléments fondamental une entrée et une sortie.

Soit $X \in \mathcal{R}^{m,n}$ l'entrée de la couche alors w est une matrice des poids de dimension (p, n) et p étant égale au nombre de neurone de la couche donc la sortie est donnée par :

$$a = \sigma(Xw^T + b)$$

Nous définissons alors z comme la sortie linéaire du modèle, c'est à dire $z = Xw^T + b$ et la partie définie $a = \sigma(z)$ regroupe les valeurs d'activations.

- Entrées :

  X : Les données d'entrée

[7]:
```
def call(self,x):
    self.z = x@self.w.T + self.b
    self.a = self.activation(self.z)
    return self.a
Couches.__call__ = call
Couches.call = call
```

Nous pouvons dés lors préciser une propriété importante : La sortie de chaque couche est l'entrée de la couche suivante.

Puis, on observe une erreur E qu'il faut minimisé, E est la fonction dite objective, c'est cette fonction qu'on cherchera à minimiser.

La descente du gradient (Algorithme d'optimisation) Donc fondamentalement, nous voulons changer les paramètres dans une couche afin que l'erreur total diminue. Il existe un moyen intelligent de faire cela qui est l'algorithme de la descente du gradient.

L'algorithme du gradient, aussi appelé algorithme de descente de gradient, désigne un algorithme d'optimisation différentiable. Il est par conséquent destiné à minimiser une fonction réelle différentiable définie sur un espace euclidien ou, plus généralement, sur un espace hilbertien.

Cette algorithme nous donne comme formule de mise à jour :

$$w = w - \alpha \frac{\partial E}{\partial w} = w - \alpha dw$$

ou le α est le pas de descente $\alpha \in [0, 1]$ et appele aussi taux d'apprentissage.

Nous devons être capable de codé pour chaque neurone (Ici c'est mon choix, vue qu'il est possible de codé la fonction hors de notre couche de neurone), une fonction d'ajustement en respectant la formule de la descente du gradient.

Cependant pour une meilleur apprentissage, nous utiliserons l'algorithme du RMSprop dans lequel l'ajustement des paramètres se fait par les formules suivantes:

$$\begin{aligned}
v_{dW} &= \beta v_{dW} + (1-\beta) dW^2 \\
v_{db} &= \beta v_{db} + (1-\beta) db^2 \\
W &= W - \alpha \frac{dW}{\sqrt{v_{dW}} + \epsilon} \\
b &= b - \alpha \frac{db}{\sqrt{v_{db}} + \epsilon}
\end{aligned}$$

.

Donc, on a comme fonction d'ajustement :

- Entrées :

  lr : Représentant la valeur de α dans la formule.

  bêta : β dans la formule.

  dW : la valeur de la dérivée de $\frac{\partial E}{\partial W}$ par rapport à la sortie finale de notre modèle

  db : la valeur de la dérivée de $\frac{\partial E}{\partial b}$ par rapport à la sortie finale de notre modèle

  epsilon : ϵ dans la formule pour stabilité numérique (pour éviter des division par 0)

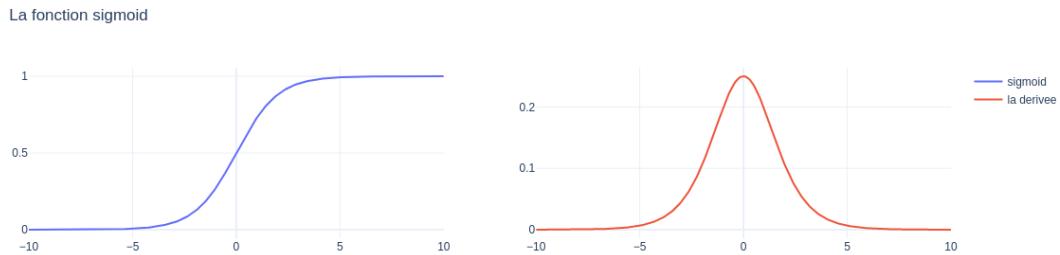
[8]:
```
def RMSprop(self,lr,beta,dW,db,epsilon=1e-10):
    # D'abord, on fait un reshape de la valeur des dérivées
    dW = dW.reshape(self.w.shape)
    db = db.reshape(self.b.shape)
    # Ensuite, on calcul
    self.vdw = beta*self.vdw + (1-beta)*np.power(dW,2)
    self.vdb = beta*self.vdb + (1-beta)*np.power(db,2)
    # Mise à jour de poids de notre modéle
    self.w -= (lr/np.sqrt(self.vdw+epsilon))*dW
    self.b -= (lr/np.sqrt(self.vdb+epsilon))*db
    return None
Couches.RMSprop = RMSprop
```

nous allons intéresser aux différents type de fonction d'activation.

Sigmoïde

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

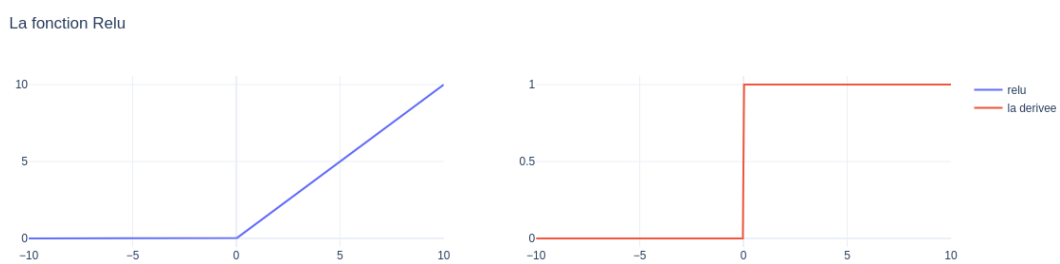
[9]:
```
t = np.linspace(-10,10,350)
#%%%%%%%%%%%%%%%%%%%%%%%----------------Logistique ou␣
↪sigmoid----------------------%%%%%%%%%%%%%%%%%%%%%%%%%%%
def sigmoid(t):
    return 1/(1+np.exp(-t))

def dsigmoid(t):
    return sigmoid(t)*(1-sigmoid(t))

fig = make_subplots(rows=1,cols=2)
trace = go.Scatter(mode='lines',x=t,y=sigmoid(t),name=r'sigmoid')
trace1 = go.Scatter(mode='lines',x=t,y=dsigmoid(t),name='la derivee')
fig.add_trace(trace,1,1)
fig.add_trace(trace1,1,2)
fig.update_layout(template='plotly_white',title_text='La fonction sigmoid')
fig.show()
```



Relu

$$f(x) = \begin{cases} 0 \text{ si } x \leq 0 \\ x \text{ sinon} \end{cases}$$

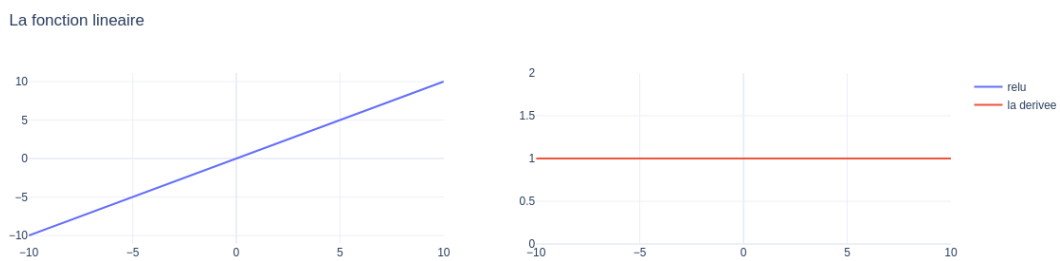
[10]:
```
t = np.linspace(-10,10,350)
###################### Relu␣
↪#########################################################
def ReLU(x):
    return np.where(x <= 0, 0, x)

def dReLU(x):
    return np.where(x <= 0, 0, 1)

fig = make_subplots(rows=1,cols=2)
trace = go.Scatter(mode='lines',x=t,y=ReLU(t),name=r'relu')
trace1 = go.Scatter(mode='lines',x=t,y=dReLU(t),name='la derivee')
fig.add_trace(trace,1,1)
fig.add_trace(trace1,1,2)
fig.update_layout(template='plotly_white',title_text=r'La fonction Relu')
fig.show()
```



Linéaire

$$f(x) = x$$

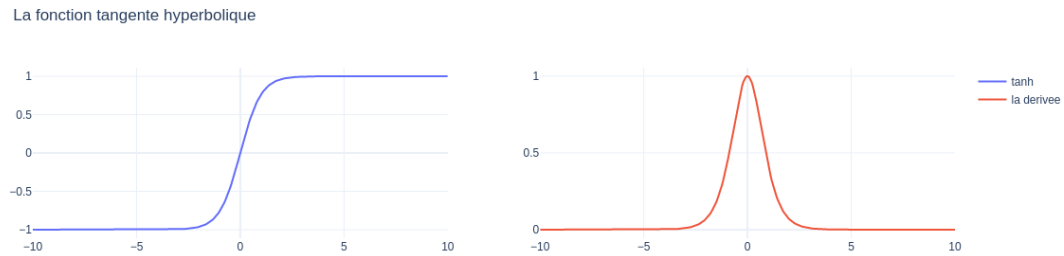
[11]:
```
t = np.linspace(-10,10,350)
################### Lineaire␣
↪#############################################################
def linear(x):
    return x

def dlinear(x):
    return np.ones_like(x)

fig = make_subplots(rows=1,cols=2)
trace = go.Scatter(mode='lines',x=t,y=linear(t),name=r'relu')
trace1 = go.Scatter(mode='lines',x=t,y=dlinear(t),name='la derivee')
fig.add_trace(trace,1,1)
fig.add_trace(trace1,1,2)
fig.update_layout(template='plotly_white',title_text='La fonction lineaire')
fig.show()
```



tanh

$$f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

[12]:
```
t = np.linspace(-10,10,350)
dtanh = Couches.Derivee(np.tanh,h=1e-8)
fig = make_subplots(rows=1,cols=2)
trace = go.Scatter(mode='lines',x=t,y=np.tanh(t),name=r'tanh')
trace1 = go.Scatter(mode='lines',x=t,y=dtanh(t),name='la derivee')
fig.add_trace(trace,1,1)
fig.add_trace(trace1,1,2)
fig.update_layout(template='plotly_white',title_text='La fonction tangente␣
↪hyperbolique')
fig.show()
```



Un petit test pour voir si l'objet ne possède pas d'erreur. Considérons une couche avec 5 neurones et une entrée à deux dimensions.

[13]:
```
a = Couches(5,activation=sigmoid,dactivation=dsigmoid)
x = np.array([[1,2],
              [2,3],
              [4,3],
              [4,5]])

y = a(x)
print(y)
print("-------")
print("shape: ", y.shape)
```

```
[[2.05716903e-01 6.38975493e-01 6.78111100e-01 4.27426351e-01
  4.06781136e-02]
 [1.34734413e-01 7.96233669e-01 7.03264195e-01 4.66393577e-01
  8.70378682e-03]
 [2.32719074e-01 9.67361271e-01 4.61035185e-01 7.46685622e-01
  8.89741577e-03]
 [5.32864270e-02 9.50117521e-01 7.49971808e-01 5.45085396e-01
  3.76320628e-04]]
-------
shape:  (4, 5)
```

On remarque la sortie est de dimension 4×5 donc chaque colonnes représentent la sortie d'un neurone.

2.3 Réseau de neurone (DNN) (Perceptron multi-couche):

Maintenant, nous allons construire une nouvelle classe d'objet qu'on va nommer NeuralNetwork.

- Entrée:

  dimension : La dimension du problème, c'est à dire le nombre de variable de notre problème.

[14]:
```
class NeuralNetwork(object):
    def __init__(self,dimension):
        # la dimension des données d'entrée, c'est a dire le nombre de variable
        self.input_shape = dimension
        # Initialisation d'une liste qui va contenir les couches de notre réseau
        self.Couches = []
        # Le nombre de couche dans notre réseau
        self.Nb_Couches = 0
        return None
```

Pour plus de liberté dans la définition du modèle de réseau de neurone, nous devons ajouter des neurones suivants les couches.

- Entrée:

  Nb_neurone: le nombre de neurone de notre couche intermédiaire

  activation: l'activation de notre couche intermédiaire

  dactivation: la dérivée de la fonction de notre fonction d'activation

[15]:
```
def Ajouter(self,Nb_neurone,activation=sigmoid,dactivation=None):
    self.Couches.append(Couches(
        Nb_neurones=Nb_neurone,
        activation=activation,
        dactivation=dactivation,
        dim_input=self.input_shape
    ))
    self.Nb_Couches += 1
    self.input_shape = Nb_neurone
    return None

NeuralNetwork.Ajouter = Ajouter
```

2.3.1 La Propaguation de l'information

Considérons maintenant l'entrée x de notre modèle, ce que l'on veut c'est avoir une sortie $\hat{y}$ et pour cela on doit faire propagé l'information x sur chaque couche. Soit un réseau de neurones contenant L couches de neurones on a :

$a_0 =$ x

Pour l dans [1,L]:

$$z_l \;=\; a_{l-1}w_l^T + b_l \;\rightarrow\; a_l = f_l(z_l) \;\rightarrow\; \hat{y} = a_L$$

Ou f_l est la fonction d'activation du couche l

[16]:
```
def Propagation(self,x):
    a = x
    for couche in self.Couches:
        a = couche(a)
```

```
    return a
NeuralNetwork.__call__ = Propagation
NeuralNetwork.Propagation = Propagation
```

2.3.2 Test de notre classe d'objet et de la propaguation de l'information.

Pour cela, nous définissons un modéle avec cinq couches neurones avec la dernière couches avec une seule neurone de sortie.

[17]:
```
modele = NeuralNetwork(dimension=2)
modele.Ajouter(Nb_neurone=6,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(Nb_neurone=16,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(Nb_neurone=1,activation=sigmoid,dactivation=dsigmoid)

x = np.array([[1,2],
              [2,3],
              [4,3],
              [4,5]])
modele(x)
```

[17]:
```
array([[0.54398127],
       [0.54356231],
       [0.54463338],
       [0.54271824]])
```

2.4 Rétro-propagation

La rétro-propagation du gradient est l'algorithme qui a fait la popularité des reseaux de neurones. Elle consiste de faire passé l'erreur de la dernière aux couches inférieurs du réseau.

On sait que pour approximer les meillieurs paramétres de notre modéle, il nous faut trouver les valeurs dW et db dans les formules de la descentes du gradient (ci-dessus). Et c'est la que la rétro-propaguaton entre en jeux.

Considérons L couches de neurones connecté entiérement les unes aux autres. Soit les W_l, b_l les poids de chaques neurones. Pour la mise a jour des poids du couche l et $\mathcal{L}$ la fonction objective de notre modéle.

Si l'on a une propaguation de l'information vers l'avant définie par :

Pour chaque couche noté l dans [1,L] dans l'ensemble des couches du reseau:

$$z_l \;=\; a_{l-1}W_l^T + b_l \text{ ou } z_l \;=\; a_{l-1}^T W_l + b_l \;\rightarrow\; a_l = f_l(z_l) \;\rightarrow\; \hat{y} = a_L$$

On posera pour la derniere couche L :

$$\frac{\partial \mathcal{L}}{\partial z_L} = \delta_L \rightarrow \frac{\partial \mathcal{L}}{\partial a_L}\frac{\partial a_L}{\partial z_L} \text{ or } a_L = \sigma_L(z_L) \rightarrow \frac{\partial a_L}{\partial z_L} = \sigma'_L(z_L)$$

$$\delta_L = \frac{\partial \mathcal{L}}{\partial a_L}\sigma'_L\left(z_L\right) \text{or } a_L = \hat{y}\delta_L = \frac{\partial \mathcal{L}}{\partial \hat{y}}\sigma'_L\left(z_L\right)$$

et pour le chouche $L-1$, on a:

$$\delta_{L-1} = \frac{\partial \mathcal{L}}{\partial z_{L-1}} = \frac{\partial \mathcal{L}}{\partial a_L}\frac{\partial a_L}{\partial z_L}\frac{\partial z_L}{\partial a_{L-1}}\frac{\partial a_{L-1}}{\partial z_{L-1}}$$
$$\text{or} \quad \frac{\partial \mathcal{L}}{\partial a_L}\frac{\partial a_L}{\partial z_L} = \delta_L$$
$$\text{donc} \quad \delta_{L-1} = \delta_L \frac{\partial z_L}{\partial a_{L-1}}\frac{\partial a_{L-1}}{\partial z_{L-1}}$$
$$\text{on sait que} \quad z_L = a_{L-1}^T W_L + b_L \Rightarrow \frac{\partial z_L}{\partial a_{L-1}} = W_L$$
$$\text{enfin on a} \quad \frac{\partial a_{L-1}}{\partial z_{L-1}} = \sigma'_{L-1}(z_{l-1})$$
$$\delta_{L-1} = \delta_L W_L \times \sigma'_{L-1}(z_{l-1})$$

avec $\times$ qui représente le produit élèment par élèment des matrices ou le produit de Hadamard

donc avec la chaine de derivée tenant compte de la valeur de δ au couche supérieur, on peut calculer la valeur de δ pour chaque couche par :

$$\delta_l = \frac{\partial \mathcal{L}}{\partial z_l} = \delta_{l+1} W_{l+1} \times \sigma'_l(z_l)$$

donc chaque couche recevra l'erreur δ.

Maintenant, on a :

$$\frac{\partial \mathcal{L}}{\partial W_L} = \frac{\partial \mathcal{L}}{\partial a_L}\frac{\partial a_L}{\partial z_L}\frac{\partial z_L}{\partial W_L} \text{ or } \frac{\partial \mathcal{L}}{\partial a_L}\frac{\partial a_L}{\partial z_L} = \delta_L$$
$$\text{donc } \frac{\partial \mathcal{L}}{\partial W_L} = \delta_L \frac{\partial z_L}{\partial W_L}$$
$$\text{or} \quad z_L = a_{L-1}^T W_L + b_L \Rightarrow \frac{\partial z_L}{\partial W_L} = a_{L-1}^T$$
$$\text{d'ou } \frac{\partial \mathcal{L}}{\partial W_L} = \delta_L a_{L-1}^T$$

donc

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{\partial \mathcal{L}}{\partial z_l}\frac{\partial z_l}{\partial W_l} \text{ or } \frac{\partial \mathcal{L}}{\partial z_l} = \delta_l$$
$$\frac{\partial \mathcal{L}}{\partial W_l} = \delta_l a_{l-1}^T$$

et pour les biais :

$$\frac{\partial \mathcal{L}}{\partial b_l} = \frac{\partial \mathcal{L}}{\partial z_l}\frac{\partial z_l}{\partial b_l}$$
$$\frac{\partial \mathcal{L}}{\partial b_l} = \delta_l$$

- Entrée:

  Input: X

  Propagation Avant: Pour l dans [1,L]: $z_l \ = \ a_{l-1} W_l^T + b_l \ \rightarrow \ a_l = f_l(z_l)$

Erreur: Calculer $\delta L = \frac{\partial \mathcal{L}}{\partial a_L} \sigma_L^{'}(z_L)$

Propagation de l'erreur : Pour l dans [L,1] $\delta_l = \delta_{l+1} W_{l+1} \times \sigma_l'(z_l)$

Gradient: Pour l dans [1,L]: $\frac{\partial \mathcal{L}}{\partial W_l} = \delta_l a_{l-1}^T$ et $\frac{\partial \mathcal{L}}{\partial b_l} = \delta_l$

[18]:
```
def RetroPropagation(self,x,y,Gcfun):
    a = self(x)
    self.Couches[-1].delta = Gcfun(a,y)*self.Couches[-1].dactivation(self.
 ↪Couches[-1].z)
    for l in reversed(list(range(self.Nb_Couches-1))):
        w = self.Couches[l+1].w; delta = self.Couches[l+1].delta
        dactivation = self.Couches[l].dactivation ; z = self.Couches[l].z
        self.Couches[l].delta = delta@w*dactivation(z)

    a = x
    for couche in self.Couches:
        dw = np.dot(couche.delta.T,a)
        db = couche.delta.sum(axis=0)
        couche.dw = dw
        couche.db = db
        a = couche.a
    return None
NeuralNetwork.RetroPropagation = RetroPropagation
```

On pourra prendre comme fonction objective:

Log loss:

$$\mathcal{L}(\hat{y}, y) = -\sum_i^d \left[y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \right]$$

MSE (mean squared errors):

$$\mathcal{L}(\hat{y}, y) = \sum_i^d (\hat{y}_i - y_i)^2$$

[19]:
```
#%%%%%%%%%%%%%%%%----------------------La fonction log loss ou cross␣
 ↪entropie-------------%%%%%%%%%%%%%%%%%%%
def Logloss(a,y,eps=1e-8):
    assert a.size == y.size
    N = a.size
    res = -y*np.log(a+eps) - (1-y)*np.log(1-a+eps)
    return np.sum(res)/N

def dLogLoss(a,y,eps=1e-8):
    assert a.size == y.size
    N = a.size
    res = -(y - a)/(a - a*a + eps)
```

```
    return res/N
#%%%%%%%%%%%%%%%----------------------La fonction mean squared error ou␣
↪MSE---------------%%%%%%%%%%%%%%%%%%

def mse(y,a):
    assert a.size == y.size
    return np.mean(np.power(y-a,2))

def dmse(a, y):
    assert a.size == y.size
    return 2*(a-y)/y.size;
```

2.4.1 Fonction d'entrainement de notre modele

Pour entrainer notre modéle, nous allons diviser nos données d'entrainement en lot (batch) que nous allons utiliser pour entrainer notre modéle, pour cela nous allons définir une nouvelle méthode:

- Entrée:

  input_shape: la taille de notre jeu de donnée

  batch_size: la taille d'un lot

  shuffle: Booleen qui définir si on doit mélanger les données de maniére aléatoire

[20]:
```
def batch(self,input_shape,batch_size,shuffle):
    ␣
↪#%%%%%%%%%%%%%%%%%%%%%%%----------------Melange-aleatoire-des-indices--------------%%%%%%%
    index = np.arange(input_shape)
    if shuffle:
        np.random.shuffle(index)
    folds = np.array_split(index,batch_size)
    if len(folds)>batch_size:
        folds[k] = np.vstack(folds[batch_size],folds[batch_size+1])
        folds.pop(batch_size+1)
    return folds
NeuralNetwork.batch = batch
```

Pour stabiliser notre modele, nous allons definir notre fonction qui divisera le learning rate si la fonction perte oscille.

- Entrée :

  perte: la valeur de notre fonction objective

  lr: le taux d'apprentissage

  e: la valeur de l'epoch actuelle

[21]:
```
def learning_rate(self,perte,lr,e):
    if self.loss[-1] - perte >= 0:
        return lr/2
```

```
    elif e == 0:
        return lr
    else:
        return lr
NeuralNetwork.learning_rate = learning_rate
```

Phase d'entrainement Ainsi nous tous les élèment faire fonctionné notre réseau de neurone, pour cela nous allons définir une fonction d'entrainement qui regroupera tous les fonction d'en haut. Et nous permettra de trouver les meilleurs paramétres de notre modéle.

- Entrée :

  x: la matrice des données

  y: la matrice des target

  lr: le taux d'apprentissage

  beta: revenir à la formule RMSprop

  loss: la fonction objective

  dloss: la dérivée de la fonction objective

  batch_size: la taille par lot

  verbose: Booleen pour si on doit affiché faire des print de notre d'entrainement

  shuffle: Booleen poour si on doit faire un mélange aléatoire des donnée d'entraiement

  BarreProgress: Booleen pou si on doit afficher une barre de progression

[22]:
```
def fit(self,x,y,epochs=1,lr=0.02,beta=0.
↪9,loss=Logloss,dloss=dLogLoss,batch_size=None,verbose=1,shuffle=False,BarreProgress=1):
↪
    self.y = np.array(y).reshape(-1,self.input_shape)
    self.x = np.array(x)
    self.Cfun = loss
    self.Gcfun = dloss
    self.loss = []
    if batch_size!=None and batch_size > y.size:
        raise ValueError('Pas assez de donnee ou batch_size trop grand')
    k = y.shape[0]//batch_size if batch_size!=None else 1
    self.ind_batch = self.batch(y.shape[0],k,shuffle)

    if verbose:
        print('{}'.format('='*70))
        print('| {:<20} | {:<20} | {:<20} |'.format('Epoches','Function␣
↪Perte',"Temps Ecoule"))
        print('{}'.format("="*70))
    t1 = time.time()
    for e in range(epochs):
```

```
        for b,batch_ind in enumerate(self.ind_batch):
            pred = self(self.x)
            perte = loss(pred,self.y)
            batch_x = self.x[batch_ind,:]
            batch_y = self.y[batch_ind,:]
            self.RetroPropagation(batch_x,batch_y,self.Gcfun)
            self.loss.append(perte)
            for couche in self.Couches:
                dw = couche.dw
                db = couche.db
                couche.RMSprop(self.learning_rate(perte,lr,e),beta,dw,db)
            if verbose == 0 and BarreProgress:
                Barreprogress(epochs,e,perte,b+1,k)
        t2 = time.time()
        if verbose:
            print('| {:<20g} | {:<20f} | {:<20f} |'.format(e+1,perte,t2-t1))

    self.loss = np.array(self.loss).reshape(-1,k).mean(axis=1)
    if verbose:
        print('#{}#'.format('='*68))
        print('{:>50}'.format('Apprentissage Termine !!!'))
    return None
NeuralNetwork.fit = fit
```

Prémier test de notre modéle Pour cela, nous considérons des données fictives et on définit un architecture simple pour résoudre un probléme de classification

[23]:
```
modele = NeuralNetwork(dimension=2)
modele.Ajouter(Nb_neurone=6,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(Nb_neurone=16,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(Nb_neurone=16,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(Nb_neurone=16,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(Nb_neurone=1,activation=sigmoid,dactivation=dsigmoid)

x = np.array([[1,2],
              [2,3],
              [4,3],
              [4,5]])

y = np.array([[1],
             [0],
             [1],
             [1]])

modele.fit(x,y,epochs=5)
```

```
====================================================================
| Epoches              | Function Perte       | Temps Ecoule         |
```

```
=====================================================================
| 1                    | 0.771448             | 0.000772             |
| 2                    | 0.638623             | 0.001370             |
| 3                    | 0.595511             | 0.001909             |
| 4                    | 0.577075             | 0.002731             |
| 5                    | 0.568893             | 0.003264             |
#===================================================================#
                         Apprentissage Termine !!!
```

On voit le bon fonctionnement de notre modéle par le biais de la dimunition de notre fonction objective.

On peut ajouter à notre class une nouvelle méthode qui nous permettra d'avoir le resume de notre modéle.

[24]:
```
def resumer(self):
    print('{}'.format('='*70))
    print('| {:>35}{:>31} |'.format("Perceptron Multi-Couche"," "))
    print('{}'.format('='*70))
    print('| {:<20} | {:<19} | {:<19} |'.format('Couches ',"Nombres de␣
↪neurone",' Nombres de parametre'))
    print('{}'.format('-'*70))
    nb_parametres_total = 0
    nb_neurones_total = 0
    for i in range(self.Nb_Couches):
        nb_neurones = self.Couches[i].w.shape[0]
        nb_parametres = self.Couches[i].w.size + self.Couches[i].b.size
        print('{}'.format('-'*70))
        print('| {:<20} | {:<20} | {:<20} |'.
↪format('Couche_'+str(i),str(nb_neurones),str(nb_parametres)))
        print("{}".format('-'*70))
        nb_parametres_total += nb_parametres
        nb_neurones_total += nb_neurones

    print('{}'.format('-'*70))
    print('| {:>5} :  {:<41} |'.format('Nombres de␣
↪paramatres',nb_parametres_total," "))
    print('| {:>5} :  {:<43} |'.format('Nombres de neurones',nb_neurones_total,"␣
↪"))
    print('{}'.format('-'*70))
NeuralNetwork.resumer = resumer
```

[25]:
```
modele.resumer()
```

```
=====================================================================
|              Perceptron Multi-Couche                              |
=====================================================================
| Couches              | Nombres de neurone  |  Nombres de parametre |
---------------------------------------------------------------------
```

```
-------------------------------------------------------------------
| Couche_0             | 6                    | 18                  |
-------------------------------------------------------------------
-------------------------------------------------------------------
| Couche_1             | 16                   | 112                 |
-------------------------------------------------------------------
-------------------------------------------------------------------
| Couche_2             | 16                   | 272                 |
-------------------------------------------------------------------
-------------------------------------------------------------------
| Couche_3             | 16                   | 272                 |
-------------------------------------------------------------------
-------------------------------------------------------------------
| Couche_4             | 1                    | 17                  |
-------------------------------------------------------------------
-------------------------------------------------------------------
| Nombres de paramatres :  691                                     |
| Nombres de neurones :  55                                        |
-------------------------------------------------------------------
```

2.5 Quelques test avec le theoréme universelle des reseaux de neurones

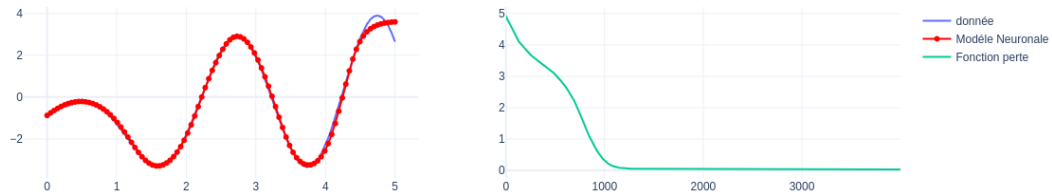
Dans la théorie mathématique des réseaux de neurones artificiels, le théorème d'approximation universelle indique qu'un réseau à propagation avant d'une seule couche cachée contenant un nombre fini de neurones (c'est-à-dire, un perceptron multicouche) peut approximer des fonctions continues sur des sous-ensembles compacts de $\mathcal{R}^n$.

[26]:
```
# Notre fonction a approximée par le biais d'un réseau de neurone
def f(x):
    return np.cos(2*x) + x*np.sin(3*x) + x**0.5 - 2

a, b = 0, 5 # intervalle [a,b]
N = 100 # taille des données
X = np.linspace(a, b, N) # abscisses
Y = f(X) # ordonnées
x_train = X.reshape(-1,1)
y_train = Y.reshape(-1,1)



# Un modéle avec un achitechture simple et dans cette exemple la derivee␣
↪numerique sera utiliser
# pour les fonction d'activations des couches intermediaires
p = 10
modele = NeuralNetwork(dimension=1)
modele.Ajouter(Nb_neurone=p,activation=np.tanh)
modele.Ajouter(Nb_neurone=p,activation=np.tanh)
modele.Ajouter(Nb_neurone=p,activation=np.tanh)
```

```
modele.Ajouter(Nb_neurone=p,activation=np.tanh)
modele.Ajouter(Nb_neurone=p,activation=np.tanh)
modele.Ajouter(Nb_neurone=1,activation=linear,dactivation=dlinear)
modele.resumer()

# Entrainement de notre modéle
modele.
 ↪fit(x_train,y_train,lr=1e-3,epochs=4000,loss=mse,dloss=dmse,verbose=0,batch_size=100)

Y1 = modele(x_train)

fig = go.Figure(layout=go.Layout(template='plotly_white'))
fig = make_subplots(rows=1,cols=2,figure=fig)
fig.add_trace(go.Scatter(x=X,y=Y,name='donnée',showlegend=True),1,1)
fig.add_trace(go.Scatter(x=X,y=Y1.ravel(),name='Modéle␣
 ↪Neuronale',showlegend=True,marker=dict(color='red'),
                         mode='markers+lines'),1,1)
fig.add_trace(go.Scatter(mode='lines',y=modele.
 ↪loss,showlegend=True,name='Fonction perte'),1,2)

fig.show()
```

```
=====================================================================
|             Perceptron Multi-Couche                               |
=====================================================================
| Couches              | Nombres de neurone  |  Nombres de parametre |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_0             | 10                  | 20                    |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_1             | 10                  | 110                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_2             | 10                  | 110                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_3             | 10                  | 110                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_4             | 10                  | 110                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_5             | 1                   | 11                    |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Nombres de paramatres :  471                                      |
```

```
| Nombres de neurones :  51                                              |
-------------------------------------------------------------------------
[==================================================] 100.0%  Fonction Objective
= 0.029083   k = 4000/4000 batches = 1/1
```



2.5.1 Estimation d'une fonction à deux dimensions

[27]:
```
# Fonction à approcher
def f(x,y):
    return x*np.cos(y)



n = 25 # pour le nb de points de l'espace discretiser
xmin, xmax, ymin, ymax = -4.0, 4.0, -4.0, 4.0

VX = np.linspace(xmin, xmax, n)
VY = np.linspace(ymin, ymax, n)
X, Y = np.meshgrid(VX, VY)
Z = f(X, Y)


# Train set pour notre modele
x_train = np.c_[X.ravel(), Y.ravel()]
y_train = Z.reshape(-1, 1)

modele = NeuralNetwork(dimension=x_train.shape[1])

modele.Ajouter(Nb_neurone=n*2,activation=ReLU, dactivation=dReLU)
modele.Ajouter(Nb_neurone=n*2,activation=ReLU, dactivation=dReLU)
modele.Ajouter(Nb_neurone=1,activation=linear,dactivation=dlinear)


modele.resumer()
```

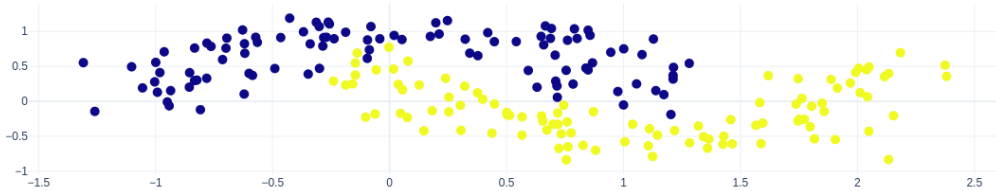
```
# Entrainement de notre modéle
modele.fit(x_train,y_train,lr=0.
 ↪01,epochs=5000,loss=mse,dloss=dmse,verbose=0,batch_size=100)


fig = go.Figure(layout=go.Layout(template='plotly_white'))
fig = make_subplots(rows=1,cols=2,figure=fig, shared_yaxes=True, specs=[[{"type":
 ↪"surface"},{"type":"surface"}]],
                    subplot_titles=("Surface d'apprentissage", "Surface estimée␣
 ↪par le modèle"))
fig.add_trace(go.Surface(x=X, y=Y, z=Z, showscale=False,showlegend=False), 1, 1)
fig.add_trace(go.Surface(x=X, y=Y, z=modele(x_train).reshape(Z.shape),␣
 ↪showlegend=False, showscale=False), 1, 2)
fig.show()
```

```
=====================================================================
|             Perceptron Multi-Couche                               |
=====================================================================
| Couches              | Nombres de neurone  |  Nombres de parametre |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_0             | 50                  | 150                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_1             | 50                  | 2550                  |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_2             | 1                   | 51                    |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Nombres de paramatres :  2751                                     |
| Nombres de neurones :  101                                        |
---------------------------------------------------------------------
[==================================================] 100.0%  Fonction Objective
= 0.027903   k = 5000/5000 batches = 6/6
```

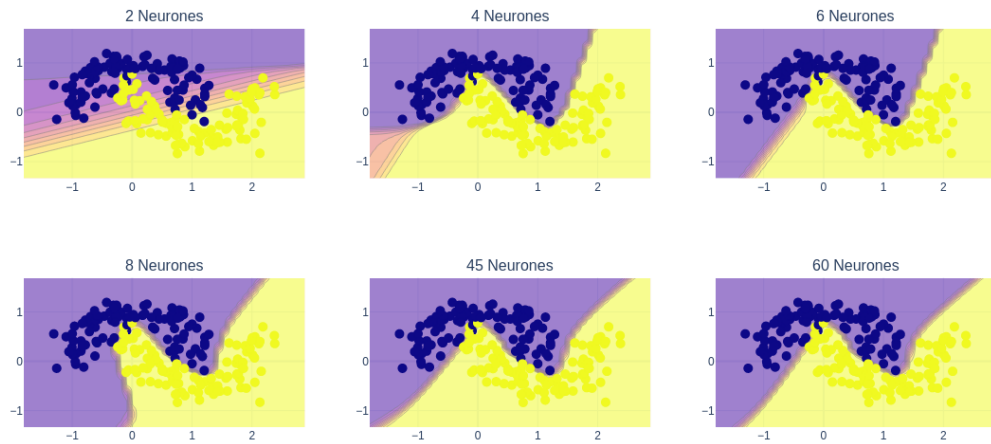
2.6 Test avec des probléme de classification réel

Maintenant, nous allons tester notre classe d'objet avec des problémes de classification de plus en plus compliquer pour appréhender son bon fonctionnnement

[28]:
```
x,y = datasets.make_moons(n_samples=200,noise=0.2)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x[:,0],y=x[:
 ↪,1],marker=dict(color=y,size=10),mode='markers'))
fig.update_layout(height=300,width=300,template='plotly_white')
fig.show()

Nb_neurones = [2,4,6,8,45,60]
i,j = 1, 1
title = tuple(str(nb)+" Neurones" for nb in Nb_neurones)
fig = make_subplots(rows=2,cols=3,subplot_titles=title)
for nb_neurone in Nb_neurones:
    modele = NeuralNetwork(dimension=2)
    modele.Ajouter(Nb_neurone=nb_neurone,activation=sigmoid)
    modele.Ajouter(Nb_neurone=1,activation=sigmoid)
    modele.
 ↪fit(x=x,y=y,epochs=5000,lr=2e-2,loss=Logloss,dloss=dLogLoss,verbose=0,BarreProgress=1)

    min_ = x.min(axis=0)
    max_ = x.max(axis=0)
    xx,yy = np.meshgrid(np.linspace(min_[0]-0.5,max_[0]+0.5,50),np.
 ↪linspace(min_[1]-0.5,max_[1]+0.5,50))
    X_ = np.c_[xx.ravel(),yy.ravel()]
    zz = modele.Propagation(X_)
    zz = zz.reshape(xx.shape)

    fig.add_trace(go.Scatter(x=x[:,0],y=x[:
 ↪,1],marker=dict(color=y,size=10),mode='markers',showlegend=False),j,i)
    fig.add_trace(go.Contour(x=xx.ravel(),y=yy.ravel(),z=zz.ravel(),opacity=0.
 ↪5,showscale=False),j,i)
    j = j+1 if i+1 == 4 else j+0
    i = i +1 if i<3 else 1

fig.update_layout(height=600,template='plotly_white')
fig.show()
```

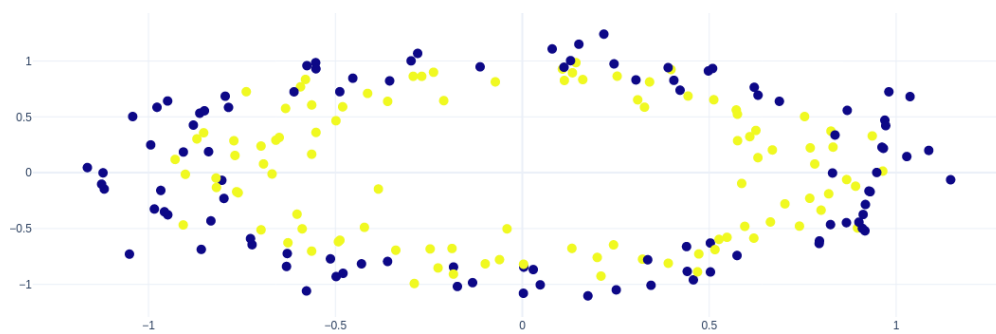


```
[==================================================] 100.0%  Fonction Objective
= 0.024582   k = 5000/5000 batches = 1/1
```

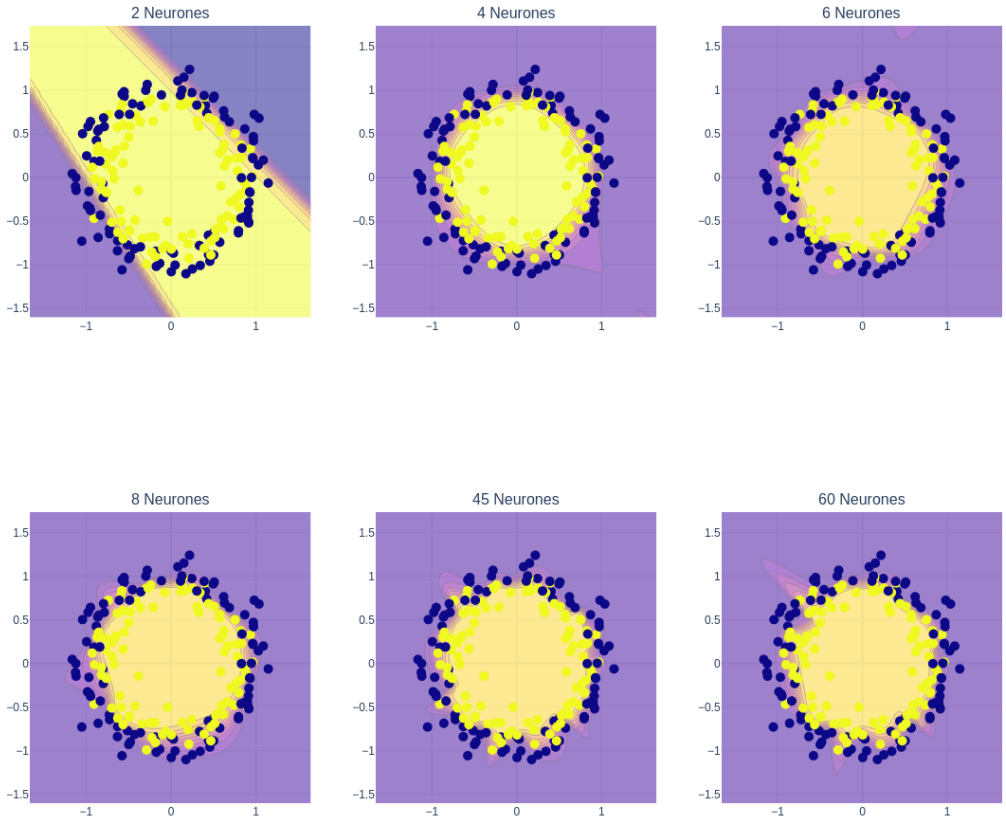


[29]:
```
x,y = datasets.make_circles(n_samples=200,noise=0.1)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x[:,0],y=x[:
 ↪,1],marker=dict(color=y,size=10),mode='markers'))
fig.update_layout(height=500,width=500,template='plotly_white')
fig.show()

Nb_neurones = [2,4,6,8,45,60]
i,j = 1, 1
title = tuple(str(nb)+" Neurones" for nb in Nb_neurones)
fig = make_subplots(rows=2,cols=3,subplot_titles=title)
for nb_neurone in Nb_neurones:
    modele = NeuralNetwork(dimension=2)
```

```
    modele.Ajouter(Nb_neurone=nb_neurone,activation=sigmoid)
    modele.Ajouter(Nb_neurone=1,activation=sigmoid)
    modele.fit(x=x,y=y,epochs=5000,lr=5e-2,loss=Logloss,dloss=dLogLoss,verbose=0)

    min_ = x.min(axis=0)
    max_ = x.max(axis=0)
    xx,yy = np.meshgrid(np.linspace(min_[0]-0.5,max_[0]+0.5,50),np.
↪linspace(min_[1]-0.5,max_[1]+0.5,50))
    X_ = np.c_[xx.ravel(),yy.ravel()]
    zz = modele.Propagation(X_)
    zz = zz.reshape(xx.shape)

    fig.add_trace(go.Scatter(x=x[:,0],y=x[:
↪,1],marker=dict(color=y,size=10),mode='markers',showlegend=False),j,i)
    fig.add_trace(go.Contour(x=xx.ravel(),y=yy.ravel(),z=zz.ravel(),opacity=0.
↪5,showscale=False),j,i)
    j = j+1 if i+1 == 4 else j+0
    i = i +1 if i<3 else 1

fig.update_layout(height=1000,template='plotly_white')
fig.show()
```




```
[==================================================] 100.0%  Fonction Objective
= 0.318750   k = 5000/5000 batches = 1/1
```



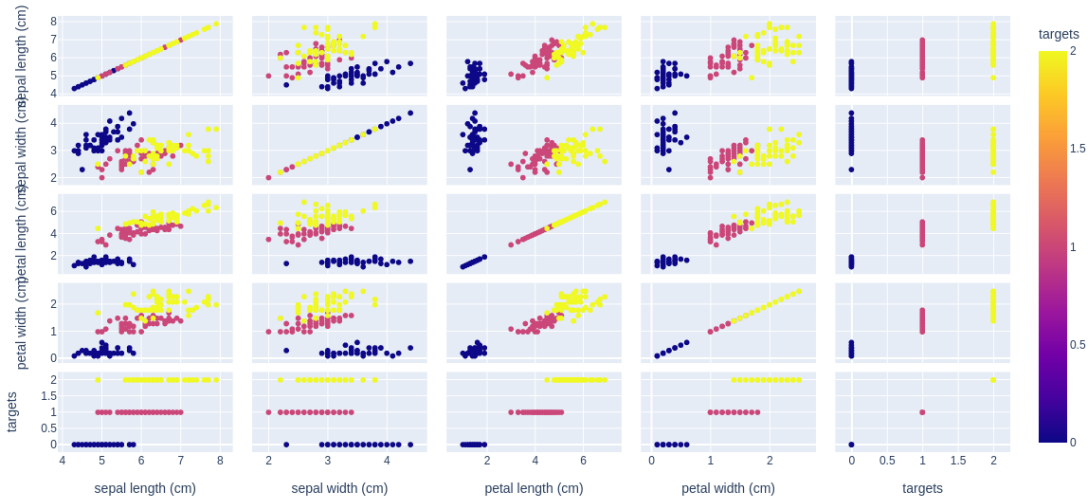
2.6.1 Application à des problémes réels de classifications

Pour démontrer le bon fonctionnement de notre modéle de deepLearning et notre classe d'objet, nous allons intéresser à des problémes de classifications.

D'abord, nous prenons comme donnée de test, les données IRIS pour la classification qui est un probléme classification

[30]:
```
from sklearn.datasets import load_iris
iris = load_iris()
data = np.hstack([iris.data,iris.target[:,np.newaxis]])
df = pd.DataFrame(data,columns=iris.feature_names+['targets'])
px.scatter_matrix(df,color='targets',height=600)
```

```
/home/seydifaye/anaconda3/envs/RL/lib/python3.10/site-
packages/plotly/express/_core.py:279: FutureWarning:

iteritems is deprecated and will be removed in a future version. Use .items
instead.
```



Maintenant, nous allons essayer de résoudre le probléme avec notre classe d'objet.

2.6.2 Préparation des données

[31]:
```
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

def to_categorical(x, input_dim=None):
    input_dim = x.unique().size if input_dim is None else input_dim
    res = np.zeros(x.shape+(input_dim, ))
    for i, v in enumerate(x.astype(int)):
        res[i, v] = 1
    return res

# On prendra alors 20% pour les données de test
x_train,x_test,y_train,y_test = train_test_split(df.drop(columns='targets'),df.
 ↪targets,test_size=0.2,random_state=1)

#--------Normalisaiton des donnees-------------#
scaler = MinMaxScaler(feature_range=[0,1])
x_train = scaler.fit_transform(x_train)
x_test = scaler.fit_transform(x_test)

#------ Pour gérer le probléme multiclasse-----#
# On préfére dans notre cas transformer nos classe sous d'un one hot encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

```
print(f"Taille train x : {x_train.shape} y : {y_train.shape}")
print(f"Taille test x : {x_test.shape} y : {y_test.shape}")
```

```
Taille train x : (120, 4) y : (120, 3)
Taille test x : (30, 4) y : (30, 3)
```

2.6.3 Definition de l'architecture de notre modele

[32]:
```
modele = NeuralNetwork(dimension=x_train.shape[1])
modele.Ajouter(64,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(64,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(3,activation=sigmoid,dactivation=dsigmoid)
modele.resumer()
```

```
=====================================================================
|             Perceptron Multi-Couche                               |
=====================================================================
| Couches              | Nombres de neurone  |  Nombres de parametre |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_0             | 64                  | 320                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_1             | 64                  | 4160                  |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_2             | 3                   | 195                   |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Nombres de paramatres :  4675                                     |
| Nombres de neurones :  131                                        |
---------------------------------------------------------------------
```

2.6.4 Entrainement du modele

[33]:
```
batch_size = 64
epochs = 2000
learning_rate = 5e-2
modele.fit(x_train,y_train,epochs=epochs,lr=learning_rate,␣
 ↪batch_size=batch_size,verbose=0,BarreProgress=1, shuffle=True)
```

```
[==================================================] 100.0%  Fonction Objective
= 0.030246   k = 2000/2000 batches = 1/1
```

2.6.5 Evaluation du modele

[34]:
```
ypred_tr = np.argmax(modele(x_train),axis=1)
ytrue_tr = np.argmax(y_train,axis=1)
ypred_ts = np.argmax(modele(x_test),axis=1)
ytrue_ts = np.argmax(y_test,axis=1)
from sklearn.metrics import confusion_matrix,classification_report
print("Rapport sur les données d'entrainement", end="\n\n")
print(classification_report(ytrue_tr,ypred_tr))
print("Rapport sur les données Test", end="\n\n")
print(classification_report(ytrue_ts,ypred_ts))
```

```
Rapport sur les données d'entrainement

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        39
           1       1.00      0.92      0.96        37
           2       0.94      1.00      0.97        44

    accuracy                           0.97       120
   macro avg       0.98      0.97      0.97       120
weighted avg       0.98      0.97      0.97       120

Rapport sur les données Test

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        11
           1       1.00      0.62      0.76        13
           2       0.55      1.00      0.71         6

    accuracy                           0.83        30
   macro avg       0.85      0.87      0.82        30
weighted avg       0.91      0.83      0.84        30
```

On voit que notre modele à des résultat assez satisfaisant, qui nous permet de confirmé le bon fonctionnement de notre modéle

2.7 Classification des donnees mnist

Nous allons maintenant teste notre modele avec un probleme plus complexe et plus reel, la classification.

[44]:
```
train_ds = pd.read_csv("./archive/mnist_train.csv", nrows=10000)
target = train_ds.pop("label")
train_ds.head()
```

[44]:

```
   1x1  1x2  1x3  1x4  1x5  1x6  1x7  1x8  1x9  1x10  ...  28x19  28x20  \
0    0    0    0    0    0    0    0    0    0     0  ...      0      0
1    0    0    0    0    0    0    0    0    0     0  ...      0      0
2    0    0    0    0    0    0    0    0    0     0  ...      0      0
3    0    0    0    0    0    0    0    0    0     0  ...      0      0
4    0    0    0    0    0    0    0    0    0     0  ...      0      0

   28x21  28x22  28x23  28x24  28x25  28x26  28x27  28x28
0      0      0      0      0      0      0      0      0
1      0      0      0      0      0      0      0      0
2      0      0      0      0      0      0      0      0
3      0      0      0      0      0      0      0      0
4      0      0      0      0      0      0      0      0

[5 rows x 784 columns]
```

[66]:

```
x_train = train_ds.to_numpy()
x_train = x_train
y_train = np.array(target, dtype=int)
x_train, x_test, y_train, y_test = train_test_split(x_train, y_train,␣
 ↪test_size=0.4)
```

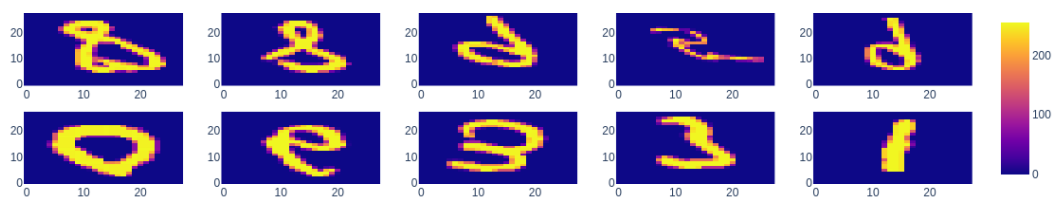
2.7.1 Architecture du modele

[78]:

```
fig = make_subplots(rows=2, cols=5)

def vectorisation(y,num_class):
    y_ = np.zeros((y.shape[0],num_class))
    for i,n in enumerate(y):
        y_[i,n] = 1
    return y_

for n, image in enumerate(x_train[:10]):
    fig1 = px.imshow(image.reshape(28, 28))
    data = fig1.data[0]
    fig.add_trace(data, row=int(n/5)+1, col=n%5+1)

fig.show()
num_class = 10


x_train =  (x_train.reshape(-1,28*28)/255).astype('float')
x_test =  (x_test.reshape(-1,28*28)/255).astype('float')
y_train = vectorisation(y_train,num_class=num_class)
y_test = vectorisation(y_test,num_class=num_class)

dim = x_train.shape[1]
```

```
modele = NeuralNetwork(dimension=dim)
modele.Ajouter(128,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(128,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(64,activation=sigmoid,dactivation=dsigmoid)
modele.Ajouter(num_class,activation=sigmoid,dactivation=dsigmoid)
modele.resumer()
modele.
 ↪fit(x_train,y_train,epochs=100,lr=2e-3,loss=Logloss,dloss=dLogLoss,verbose=0,batch_size=64,␣
 ↪BarreProgress=1)

fig = make_subplots(rows=2,cols=4,subplot_titles=("Taux␣
 ↪d'apprentissage",'Precision sur les Test','Matrice de confusion',
                                                    'F1 score',"Recall␣
 ↪Score",'Precision Score','La moyenne des scores'),
                   specs=[[{'type':'pie'},{'type':'pie'},{'colspan':2,'type':
 ↪'table'},None],
                           [{},{},{},{}]])
labels_1 = ["Taux_app",'Erreur_app']
labels_2 = ['Taux_test','Erreur_test']
taux_app = accuracy_score(y_train.argmax(axis=1),modele.Propagation(x_train).
 ↪argmax(axis=1))
taux_test =  accuracy_score(y_test.argmax(axis=1),modele.Propagation(x_test).
 ↪argmax(axis=1))
f1_score_test = f1_score(y_test.argmax(axis=1),modele.Propagation(x_test).
 ↪argmax(axis=1),average=None)
recall_score_test = recall_score(y_test.argmax(axis=1),modele.
 ↪Propagation(x_test).argmax(axis=1),average=None)
precision_score_test = precision_score(y_test.argmax(axis=1),modele.
 ↪Propagation(x_test).argmax(axis=1),average=None)
f1_moy = f1_score(y_test.argmax(axis=1),modele.Propagation(x_test).
 ↪argmax(axis=1),average='macro')
recal_moy = recall_score(y_test.argmax(axis=1),modele.Propagation(x_test).
 ↪argmax(axis=1),average='macro')
pre_moy = precision_score(y_test.argmax(axis=1),modele.Propagation(x_test).
 ↪argmax(axis=1),average='macro')
mat_conf = confusion_matrix(y_test.argmax(axis=1),modele.Propagation(x_test).
 ↪argmax(axis=1))
header = [['']]+[[f"{num}"] for num in range(10)]
cells = [[f"{num}" for num in range(10)]]+[[mat_conf[j,i] for i in range(10)]␣
 ↪for j in range(10)]
err_app = 1-taux_app
err_test = 1-taux_test
fig.add_trace(go.Pie(labels=labels_1,values=[taux_app,err_app],hole=0.
 ↪6,textposition='inside',textinfo='percent+label',
                     showlegend=False,pull=[0,0.3]),1,1)
```

```
fig.add_trace(go.Pie(labels=labels_2,values=[taux_test,err_test],hole=0.
↪6,textposition='inside',textinfo='percent+label',
                    showlegend=False,pull=[0,0.3]),1,2)
fig.add_trace(go.Table(header=dict(values=header),cells=dict(values=cells)),1,3)
fig.add_trace(go.Bar(x=['{}'.format(num) for num in range(10)],
                     y=f1_score_test,text=["{:.2f}%".format(100*s) for s in␣
↪f1_score_test],showlegend=False),2,1)
fig.add_trace(go.Bar(x=['{}'.format(num) for num in range(10)],
                     y=recall_score_test,text=["{:.2f}%".format(100*s) for s in␣
↪recall_score_test],showlegend=False),2,2)
fig.add_trace(go.Bar(x=['{}'.format(num) for num in range(10)],
                     y=precision_score_test,text=["{:.2f}%".format(100*s) for s␣
↪in precision_score_test],
                     showlegend=False),2,3)
fig.add_trace(go.Bar(x=['F1 Score','Recall Score','Precision '],
                     y=[f1_moy,recal_moy,pre_moy],
                     text=["{:.2f}%".format(100*s) for s in␣
↪[f1_moy,recal_moy,pre_moy]],showlegend=False),2,4)
fig.update_layout(template='plotly_white',height=600,title_text='Rapport de␣
↪classification sur les testes')
fig.show()
```



```
=====================================================================
|              Perceptron Multi-Couche                              |
=====================================================================
| Couches              | Nombres de neurone  |  Nombres de parametre |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_0             | 128                 | 100480                |
---------------------------------------------------------------------
---------------------------------------------------------------------
| Couche_1             | 128                 | 16512                 |
---------------------------------------------------------------------
---------------------------------------------------------------------
```

```
| Couche_2              | 64                    | 8256                  |
-----------------------------------------------------------------------
-----------------------------------------------------------------------
| Couche_3              | 10                    | 650                   |
-----------------------------------------------------------------------
-----------------------------------------------------------------------
| Nombres de paramatres :  125898                                     |
| Nombres de neurones :  330                                          |
-----------------------------------------------------------------------
[==================================================] 99.0%  Fonction Objective =
0.000014   k = 100/100 batches = 93/93
```

On peut voir que le modele fait du surapprentissage, qu'il faudrait diminuer le nombre d'epochs ou ajouter de la regularisation pour avoir de meilleur resultat.

2.8 Conclusion

Ce tutorielle à pour but de demistifier le boite noire que consiste qu'est les reseaux de neurones et de nous donner une meilleur comprehension des parametre et des fonctions d'activations.

Les classes d'objet conçuent pour ce tuto peuvent etre ameliorer.